



MIRADOR 360°

(DAS) Documento Arquitectura de Software
Versión 1.0

Identificación de Documento

Identificación	Grupo 2
Proyecto	Mirador 360°
Versión	1.1
Documento mantenido por	Grupo 2
Fecha de última revisión	-
Fecha de próxima revisión	-
Documento aprobado por	
Fecha de última aprobación	

Historia de Revisiones

Fecha	Versión	Descripción	Autor
09/04/2026	1.1	Levantamiento de requerimientos	Grupo 2

Tabla de Contenidos

1. INTRODUCCIÓN	4
1.1. CONTEXTO DEL PROBLEMA	4
1.2. PROPÓSITO	4
1.3. ÁMBITO	4
1.4. DEFINICIONES, ACRÓNIMOS Y ABREVIACIONES	4
1.5. RESUMEN EJECUTIVO	4
1.6. ARQUITECTURA DEL SISTEMA	4
2. VISIÓN DEL SISTEMA	4
2.1. DESCRIPCIÓN GENERAL DEL SISTEMA	4
2.2. OBJETIVOS DEL SISTEMA	4
2.3. REQUERIMIENTOS FUNCIONALES	4
2.4. REQUERIMIENTOS NO FUNCIONALES	5
2.5. SUPUESTOS Y DEPENDENCIAS	5
3. ESTILOS Y PATRONES ARQUITECTÓNICOS	5
3.2. JUSTIFICACIÓN DEL ESTILO SEGÚN EL CONTEXTO DEL SISTEMA	5
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS	5
4.1. VISTA DE ESCENARIO	5
4.1.1. <i>Propósito</i>	5
4.1.2. <i>Actores</i>	5
4.1.3. <i>Diagrama general de casos de uso</i>	5
4.1.4. <i>Diagrama de casos de uso específicos</i>	5
4.1.6. <i>Especificación de casos de uso</i>	5
4.2. VISTA LÓGICA	6
4.2.1. <i>Propósito</i>	6
4.2.2. <i>Diagrama de clases</i>	6
4.2.3. <i>Descripción diagrama de clases</i>	6
4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO	7
4.3.1. <i>Propósito</i>	7
4.3.2. <i>Diagrama de componente</i>	7
4.3.3. <i>Descripción diagrama de componente</i>	7
4.3.4. <i>Diagrama de paquete</i>	7
4.3.5. <i>Descripción diagrama de paquete</i>	7
4.4. VISTA DE PROCESOS	7
4.4.1. <i>PROPÓSITO</i>	7
4.4.2. <i>DIAGRAMA DE ACTIVIDAD</i>	7
4.4.3. <i>DESCRIPCIÓN DIAGRAMA DE ACTIVIDAD</i>	7
4.5. VISTA FÍSICA	7
4.5.1. <i>Propósito</i>	7
4.5.2. <i>Diagrama de despliegue</i>	7
4.5.3. <i>Descripción diagrama de despliegue</i>	7
5. REQUISITOS DE CALIDAD	7

5.1. PROPÓSITO	7
5.3. <i>Reglas y criterios de evaluación de calidad</i>	7
6. PRINCIPIOS DE DISEÑO APLICADOS	8
6.1. <i>Propósito</i>	8
6.2. PRINCIPIOS DE DISEÑO (POR EJEMPLO: ABSTRACCIÓN, ACOPLAMIENTO, COHESIÓN, ENCAPSULAMIENTO, MODULARIDAD)	8
7. PROTOTIPO	8
7.1. PROPÓSITO	8
7.2. MOCKUPS (IMÁGENES CON UNA BREVE DESCRIPCIÓN)	8
7.3. JUSTIFICAR HERRAMIENTAS DE PROTOTIPADO	8
8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	8
8.1. PROPÓSITO	8
8.2. LISTA DE VERIFICACIÓN	8
8.3. ANÁLISIS Y MÉTRICAS DE RESULTADOS	8
9. CONTROL DE VERSIONES	8
9.1. PROPÓSITO	8
9.2. CONTROL DE VERSIÓN UTILIZADO (JUSTIFICAR EL TIPO DE CONTROL DE VERSIÓN UTILIZAD (FECHA, SEMÁNTICA O SECUENCIAL)	8
9.3. JUSTIFICAR HERRAMIENTAS DE VERSIONAMIENTO	8
7. CONCLUSIONES	8
8. BIBLIOGRAFÍA	8
9. ANEXOS	8
9.1. CARTA GANTT	8

1. INTRODUCCIÓN

1.1. Contexto del Problema

Actualmente nos centraremos en el edificio “El Mirador” el cual se encuentra en un escenario desfavorable para su administración por la mala gestión de cobros y transparencia de este mismo ámbito. Presenta varios problemas importantes, principalmente todo el papeleo se realiza a mano, sin programas ni documentos de acceso online, lo que produce errores humanos, atraso con las cuotas, lo que repercute en una mala gestión del mantenimiento y transparencia en las cuentas de este edificio.

1.2. Propósito

En este documento se dejará por escrito y de forma detallada cómo va a estar armado e implementado este nuevo sistema. Desde el equipo a cargo de la programación hasta el usuario final, dando a entender el funcionamiento y lo que se hará durante el proyecto.

1.3. Ámbito

Mirador 360 consiste en la creación e implementación de una página web tipo responsive para que cada actor del edificio tenga acceso a ella, otorgando los permisos correspondientes a cada uno según la necesidad del problema. En la página estará disponible toda la información correspondiente y de utilidad para llevar una cuenta clara de los ingresos y egresos de dinero, además de una sección de reclamos y sugerencias/solicitudes con la mayor transparencia posible.

1.4. Definiciones, acrónimos y abreviaciones

ACRÓNIMO	DESCRIPCIÓN
<i>RESPONSIVE</i>	Diseño de desarrollo web donde el sitio se adapta a cualquier tamaño de pantalla, tablet, computadora, móvil, etc.
<i>MVC</i>	Modelo-Vista-Controlador, tipo de patrón de arquitectura de software que separa una aplicación en 3 partes, mejora la organización y mantenimiento del código
<i>Base de Datos o BD</i>	Es donde se almacena toda la información correspondiente al edificio, nombres, cobros, solicitudes, errores

1.5. Resumen ejecutivo

Básicamente, se busca crear e implementar un sistema automático que reemplace al actual para evitar los problemas que se han generado por el deficiente manejo del sistema de cobro. La idea es que la gente pague a tiempo, las cuentas claras para ambas partes y que la administración no pierda la paciencia y el tiempo cobrando cada que ve un moroso, mejorando así la convivencia del edificio.

1.6. Arquitectura del sistema

Trabajaremos con una arquitectura basada en el patrón MVC (Modelo-Vista-Controlador). El cual nos permite separar en bloques la información que queremos que se muestre a los usuarios finales, para que finalmente en la pantalla de usuario se muestre solo la información relevante para ellos. Además mejora el mantenimiento y corrección de errores del sistema.

2. VISIÓN DEL SISTEMA

2.1. Descripción general del sistema

El sistema será una página web en donde cada usuario podrá ingresar con su cuenta previamente creada, usuario-contraseña. Dentro del sistema los usuarios tendrán acceso a su información personal. Por otro lado, la administración tendrá acceso especial para ver los informes detallados del dinero que entra y sale, y saber de forma inmediata los usuarios morosos o cualquier otra información relevante.

2.2. Objetivos del sistema

- Objetivo general: “Desarrollar una solución tecnológica para el edificio *El mirador* para gestionar las ganancias y gastos de este, asegurando un proceso transparente y mejorando la eficiencia de cobros y reparaciones”
- Objetivos específicos :
 - Registrar la información de los usuarios en una base de datos
 - Realizar cálculos y registros de pagos
 - Almacenar las solicitudes y mantenciones del edificio
 - Generar reportes relacionados a deudas o pagos

2.3. Requerimientos funcionales

1. Gestionar el personal: el sistema debe gestionar de manera eficiente todo usuario relacionado al edificio, incluyendo datos como nombre, contacto y/o número de departamento.
2. Gestionar los pagos: el sistema se encargará de almacenar los registros de los pagos y realizar los cálculos necesarios para realizar todas las operaciones de relevancia.
3. Consulta de deudas: el sistema le dará acceso a los administradores para visualizar a usuarios que presenten deudas pendientes.
4. Registro de solicitudes: el sistema debe permitir ingresar a los usuarios las solicitudes o reclamos en relación al edificio.
5. Consulta de estados de solicitudes: el sistema debe permitir a los usuarios realizar consultas con respecto a las solicitudes ingresadas.
6. Gestión de mantenimiento: el sistema debe llevar un registro sobre las tareas de mantenimiento, marcadas como realizadas o pendientes.
7. Notificaciones a usuarios: el sistema debe notificar de manera

oportuna sobre deudas pendientes o información relevante del edificio.

8. Generación de reportes: el sistema debe generar reportes en base a pagos, deudas, solicitudes y mantenimiento del edificio para el administrador.

2.4. Requerimientos no funcionales

1. El sistema debe registrar de manera segura la información de los usuarios.
2. El sistema debe permitir iniciar sesión a los usuarios.
3. El sistema debe restringir el acceso acorde a los roles del edificio.
4. El sistema debe cerrar sesión de usuarios tras periodo de inactividad.
5. El sistema debe responder a las solicitudes en menos de 2 segundos.
6. El sistema debe soportar a múltiples usuarios simultáneamente.
7. El sistema debe tener una interfaz amigable e intuitiva, fácil de usar.
8. El sistema debe ser de tipo responsive.
9. El sistema debe estar disponible el 99% del tiempo
10. El código del sistema debe permitir modificaciones sin afectar otras funcionalidades.
11. El sistema debe evitar duplicidad de datos y un correcto ingreso de estos.
12. El sistema debe ser compatible con navegadores modernos.
13. El sistema debe ser compatible con diferentes sistemas operativos.
14. El sistema debe enviar notificaciones oportunas.
15. El sistema debe realizar copias de seguridad periódicamente.
16. El sistema debe permitir recuperar datos ante fallos.

2.5. Supuestos y dependencias

- Supuestos: Asumimos que los vecinos tienen acceso a un teléfono o computador con internet para poder meterse al sistema.
- Dependencias: El sistema va a depender de una integración con alguna pasarela de pago externa (como Mercado Pago o Webpay) para poder procesar las transacciones de forma automática y segura.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1. Estilo arquitectónico adoptado

Para este proyecto se utilizará Arquitectura Monolítica en Capas , ejecutado mediante un modelo Cliente-Servidor. Siguiendo los lineamientos de la norma ISO/IEC/IEEE 42010 para la descripción de arquitecturas de software.

3.2. Justificación del estilo según el contexto del sistema

Este estilo organiza el sistema en una jerarquía de capas, donde cada capa tiene una responsabilidad específica y se comunica únicamente con la capa inmediatamente inferior. Además facilita el encapsulamiento de la complejidad del cálculo y procesamiento de los gastos que realiza el software, así también, ocultando la capa por ejemplo de la base de datos en la capa de presentación. Esto ayuda a reducir errores en cadena, permitiendo modificar módulos específicos evitando comprometer la estabilidad del sistema.

Esta arquitectura promueve una alta cohesión, donde cada capa tiene una responsabilidad única y bien definida, y un bajo acoplamiento, minimizando las dependencias directas entre componentes.

Se evaluó la posibilidad de utilizar arquitectura de microservicios por la integración de plataformas externas (para pagos y notificaciones), sin embargo, dada la escala del proyecto, es decir la cantidad de usuario a la que nos enfrentamos, los microservicios agregarían una complejidad operacional e infraestructural innecesaria. La arquitectura en capas centraliza el procesamiento en una única base de datos transaccional

3.3. Patrones de diseño aplicados

Para el caso de Mirador 360 establecimos un patrón de diseño MVC y repositorio, para así poder separar lo que es la interfaz de la lógica, además abstraer la persistencia y centralización del acceso a los datos.

4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS

4.1. VISTA DE ESCENARIO

4.1.1. Propósito

El objetivo de este diagrama es dar a conocer gráficamente el comportamiento del sistema, validando su estructura y cómo satisface los requerimientos desde una perspectiva como usuarios finales. Es el que unifica y da sentido a las demás vistas.

4.1.2. Actores

Actores diagrama general caso de uso:

- Administración
- Residente (dueño/arrendatario)
- Personal del edificio
- Sistema de pago
- Sistema de notificaciones
- Comité administrativo

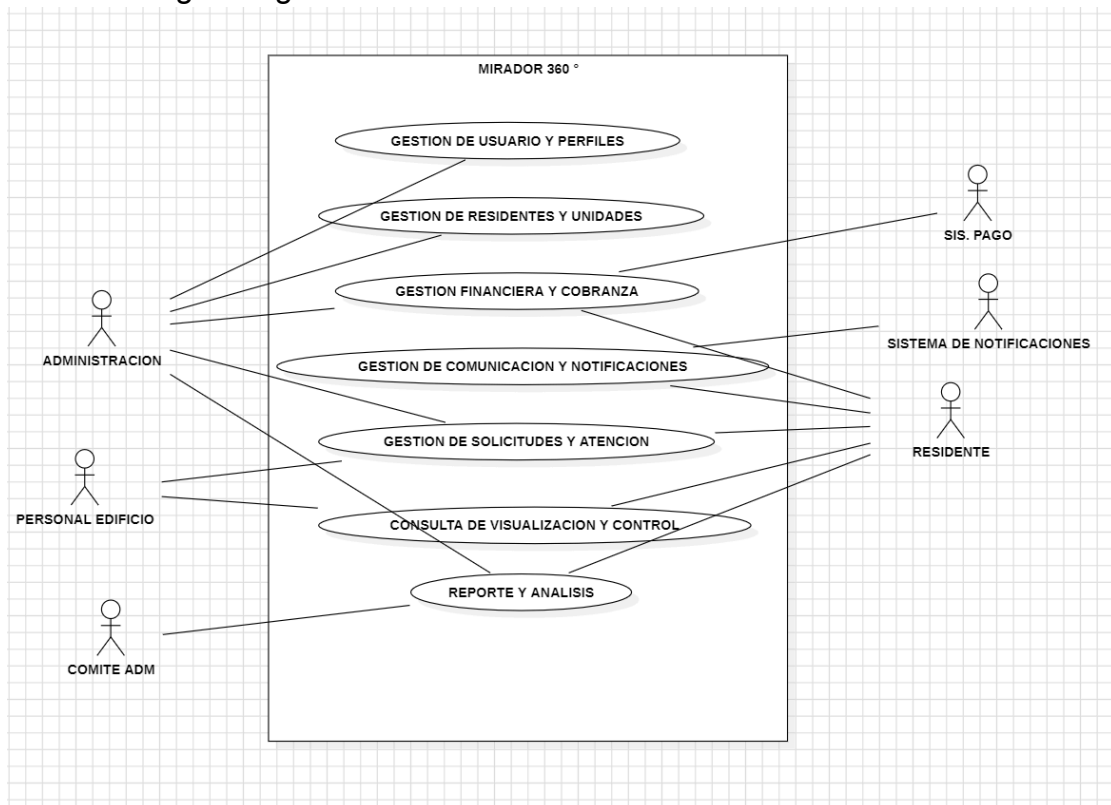
Actores diagrama intermedio caso de uso:

- Administración
- Residente (dueño/arrendatario)
- Sistema de pago

Actores diagrama específico caso de uso:

- Administración
- Residente (dueño/arrendatario)
- Sistema de pago

4.1.3. Diagrama general de casos de uso



4.1.4. Diagrama de casos de uso intermedio y específico

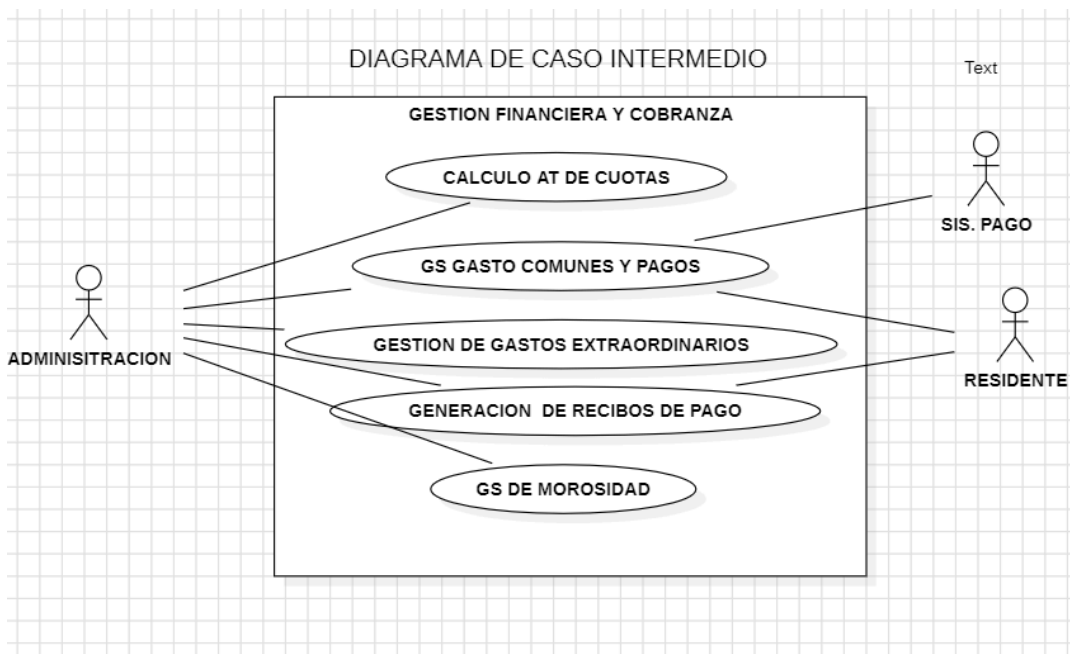
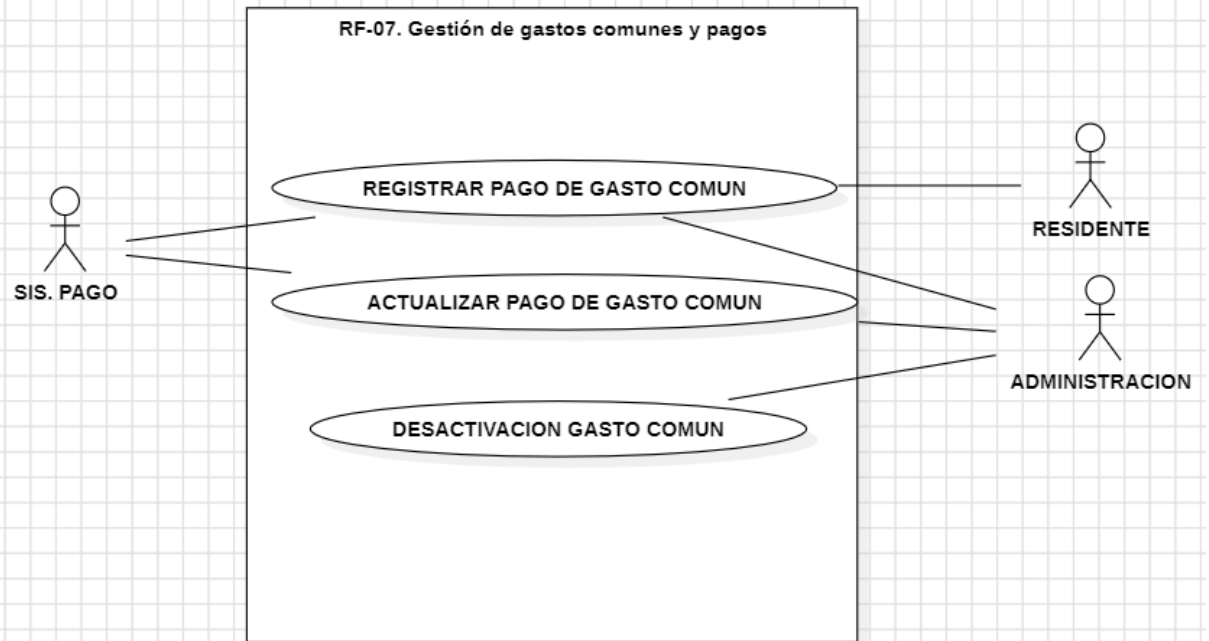


DIAGRAMA CASO ESPECIFICO



4.1.5. Lista de casos de uso

Código	Nombre	Actores
CU-001-001	GESTIÓN DE USUARIO Y PERFILES	administración
CU-001-002	GESTIÓN DE RESIDENTES Y UNIDADES	administración
CU-001-003	GESTIÓN FINANCIERA Y COBRANZA	administración, sis. pago, residente
CU-001-004	GESTION DE COMUNICACION Y NOTIFICACIONES	sis. notificaciones, residentes
CU-001-005	GESTIÓN DE SOLICITUDES Y ATENCIÓN	administración, personal edificio, residente
CU-001-006	CONSULTA DE VISUALIZACIÓN Y CONTROL	residente personal edificio
CU-001-007	REPORTE Y ANÁLISIS	administración, comité edificio, residente
CU-002-001	CALCULO AT DE CUOTAS	administración
CU-002-002	GS GASTO COMUNES Y PAGOS	administración, sis. pago, residente
CU-002-003	GESTIÓN DE GASTOS EXTRAORDINARIOS	administración
CU-002-004	GENERACIÓN DE RECIBOS DE PAGO	administración, residentes
CU-002-005	GESTIÓN DE MOROSIDAD	administración
CU-003-001	REGISTRAR PAGO DE GASTO COMÚN	sis. pago, residente, administración
CU-003-002	ACTUALIZAR PAGO DE GASTO COMÚN	sis. pago, administración
CU-003-003	DESACTIVACIÓN GASTO COMÚN	administración

4.1.6. Especificación de casos de uso

Caso de Uso	Gestión de gastos comunes	Identificador:
--------------------	---------------------------	-----------------------

	y pagos	CU-002-002
Actores	administración - sistema de pago - residente	
Tipo	Caso de uso específico	
Referencias	-Gestión de residentes y unidades -Gestión financiera y cobranza	
Precondición	- El Residente debe estar autenticado en la plataforma MIRADOR 360° -El Administrador debe haber ejecutado previamente el proceso de "Emisión de Cobros" para el periodo en curso. -El sistema debe tener conexión activa con la API de la Pasarela de Pagos externa.	
Postcondición	La deuda del residente queda en \$0 para el periodo seleccionado, se actualiza la deuda y existe un registro del pago asociado al departamento.	
Descripción	Describe el flujo integral que permite a un Residente consultar el estado financiero de su departamento	
Resumen	El objetivo fundamental de este módulo es automatizar el ciclo financiero del edificio, garantizando transparencia, integridad de datos y facilidad de uso para los residentes, lo que impacta directamente en la reducción de la tasa de morosidad.	

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
1	Residente	El residente accede al módulo "Mis pagos"
2	Sistema	El sistema consulta en la BD, calculando el total de deuda incluyendo morosidad si aplica
3	Residente	El residente selecciona las deudas que desee pagar
4	Sistema	El sistema genera un ID de transacción único y redirige al servicio de pago externo
5	Residente	El residente ingresa con credenciales bancarias y confirma el pago
6	Sistema	El servicio de pago retorna el ID exitoso
7	Sistema	El sistema valida el ID y registra los pagos realizados
8	Sistema	El sistema envía mensaje de éxito agregando una opción de descargar comprobante

CURSO ALTERNATIVO

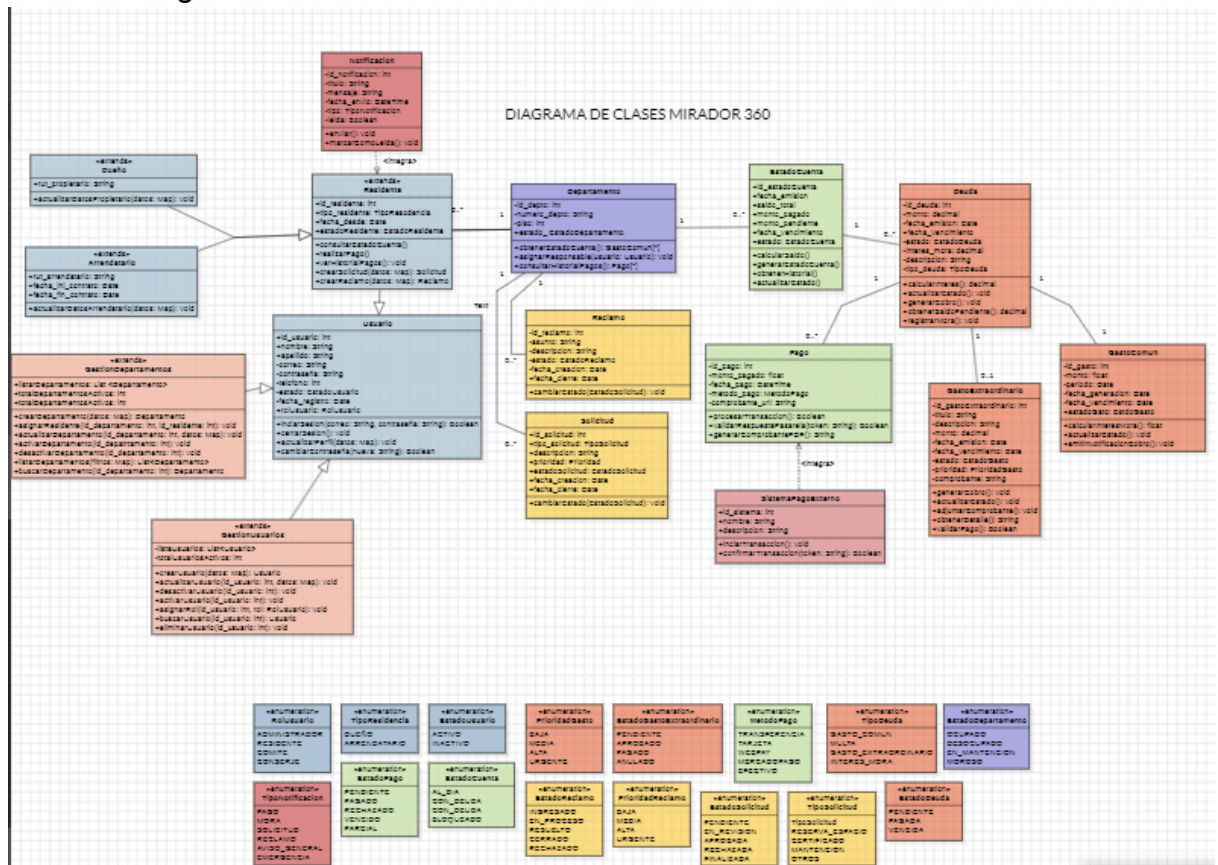
Nro.	Descripción de acciones alternas
5.1	Pago rechazado por entidad bancaria, el sistema envía al usuario nuevamente a la selección de deudas y solicita que realice el proceso nuevamente.
5.2	Se pierde conexión con el sistema de pago, el sistema envía al usuario nuevamente a la selección de deudas y solicita que realice el proceso nuevamente.

4.2. VISTA LÓGICA

4.2.1. Propósito

Este diagrama de clases modela los conceptos del dominio del negocio, sus atributos, los comportamientos (métodos) y las asociaciones que garantizan la integridad de los datos y el cumplimiento de los requerimientos funcionales descritos en él.

4.2.2. Diagrama de clases



4.2.3. Descripción diagrama de clases

El sistema se organiza a partir de la clase principal Usuario, desde la cual se deriva la clase Residente, subdividida posteriormente en Dueño y Arrendatario. El diagrama incorpora relaciones de asociación, herencia y multiplicidad para representar correctamente la interacción entre las entidades del sistema.

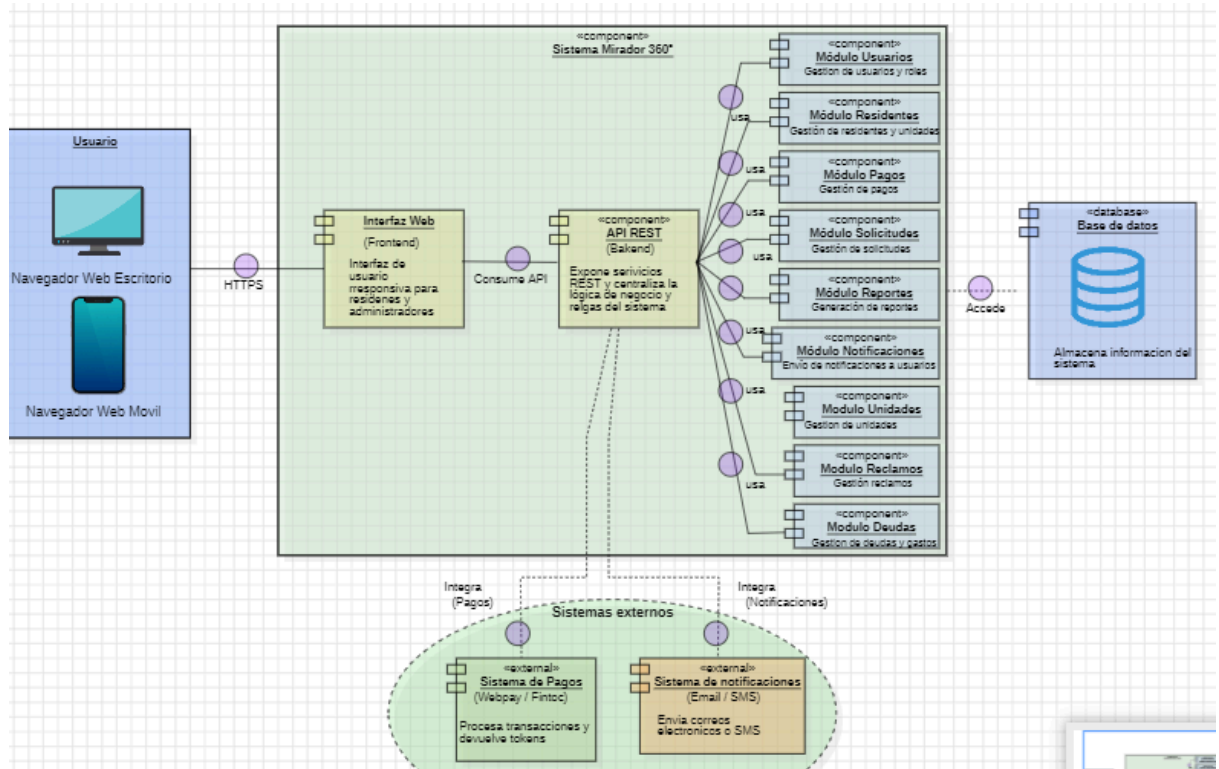
También se incluyen enumeraciones para representar estados y tipos específicos, permitiendo mejorar la organización y mantenibilidad del modelo.

4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO

4.3.1. Propósito

El diagrama representa la arquitectura modular del sistema y la interacción entre los distintos componentes internos y externos o servicios. Además permite visualizar cómo se organiza funcionalmente la solución y cómo se comunican sus módulos principales.

4.3.2. Diagrama de componente (TODO EL PROY)



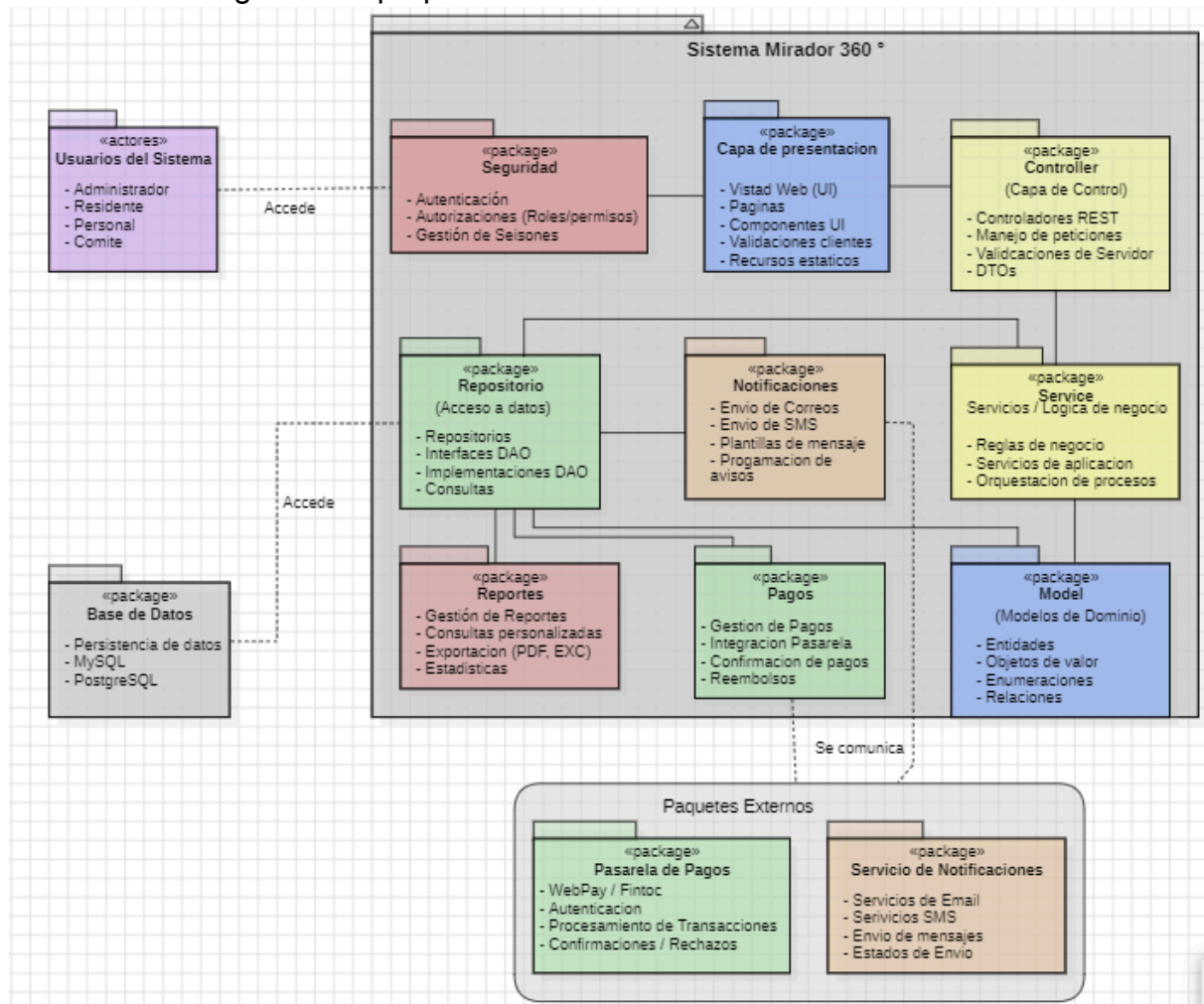
4.3.3. Descripción diagrama de componente

Como hemos comentado anteriormente el sistema Mirador 360° se basa en una estructura de multicapa, como podemos observar está compuesta por:

- Interfaz Web
- API rest
- Módulos internos
- Base de datos
- Sistemas externos, sistema de pago y notificación

Si hablamos del backend, podemos observar que está representado por una API REST la cual comunica cada módulo en donde cada uno desempeña tareas específicas y centraliza la lógica y reglas de negocio.

4.3.4. Diagrama de paquete



4.3.5. Descripción diagrama de paquete

Aquí el diagrama se organiza en paquetes, para poder representar la lógica de negocio y mostrar qué tarea realiza cada uno además de cómo se comunican.

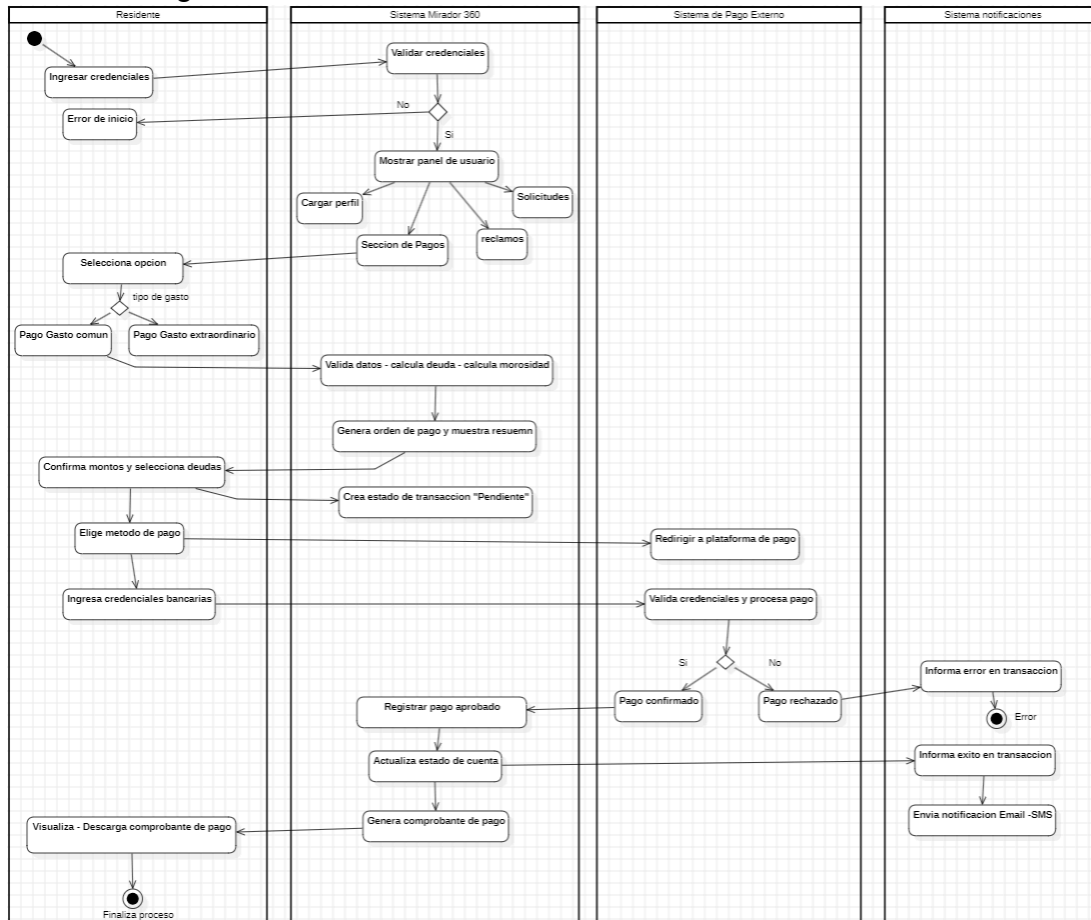
Cada paquete agrupa funcionalidades relacionadas, administrar las solicitudes y respuestas del sistema, representar las entidades del dominio, modelar las integraciones externas y gestionar el acceso a los datos.

4.4. VISTA DE PROCESOS

4.4.1. Propósito

La Vista de Procesos describe el comportamiento dinámico del sistema mediante el diagrama de actividad del caso de uso específico seleccionado, CU-003-02-01, el cual es Registrar Pago de Gasto Común (RF-07). Representa el flujo de trabajo con sus acciones, decisiones y ramificaciones.

4.4.2. Diagrama de actividad



4.4.3. Descripción diagrama de actividad

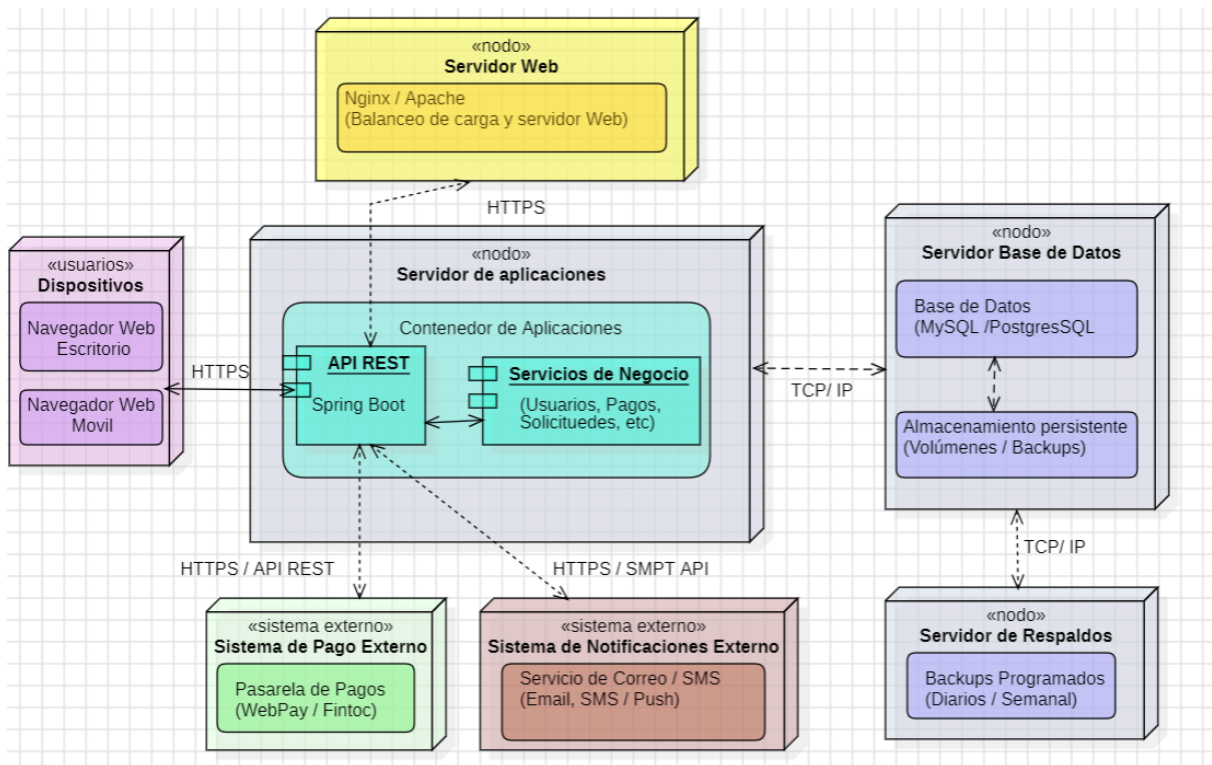
El proceso inicia con el primer paso que es realizado por el residente, donde ingresa sus credenciales y el sistema las debe validar, de ser erróneo entrega un error de inicio. Luego, si es válido, se muestra el panel de usuario en donde se selecciona la sección de pagos y se escoge el tipo de pago a cancelar. Posteriormente el sistema realiza los cálculos pertinentes y el usuario selecciona las deudas a cancelar. Se redirige al servicio externo de pago y este validar sus credenciales, si el pago es exitoso, se actualiza el estado de la cuenta del usuario, en paralelo envía notificación, y entrega la orden de pago para opcionalmente la descarga de comprobante, de ser invalidadas las credenciales, se envía notificación de error.

4.5. VISTA FÍSICA

4.5.1. Propósito

En la vista física nos enfocamos en mostrar la infraestructura y tecnología como tal en donde se ejecutará el sistema. Se pueden visualizar los nodos

4.5.2. Diagrama de despliegue



4.5.3. Descripción diagrama de despliegue

En el despliegue se contemplan los usuarios, los servidores de aplicación, las bases de datos y sistemas externos.

El servidor se conecta con los usuarios mediante HTTPS y así mismo con los sistemas externos, implementando API REST para pagos y SMPT API para las notificaciones. También se muestran los servidores de respaldo para la base de datos, en donde se encuentran backups programados, para respaldar completamente la información valiosa del sistema.

5. REQUISITOS DE CALIDAD

5.1. Propósito

La finalidad de los requisitos es garantizar que el sistema Mirador 360° entregue una solución segura, confiable, mantenible y eficiente para todos los vecinos y actores del edificio. Permitiendo definir los criterios técnicos y funcionales que aseguren la mejor experiencia de usuario posible, adecuándose a las necesidades de cada representante del sistema, además de facilitar la escalabilidad y mantenimiento a futuro de este.

El cumplimiento de los requisitos de calidad permitirá un funcionamiento estable, seguro y organizado, considerando la gestión de usuarios, departamentos, pagos, reclamos, solicitudes y notificaciones.

5.2. Atributos de calidad

ATRIBUTO DE CALIDAD	DESCRIPCIÓN	JUSTIFICACIÓN
Usabilidad	Un sistema intuitivo, fácil de usar y comprender cada sección	Este será utilizado por distintos usuarios, con conocimiento básico de aplicaciones, por lo que es necesario una interfaz que facilite la interacción usuario/sistema
Rendimiento	El sistema debe tener tiempos de respuestas rápidas ante solicitudes de usuarios	Los procesos deben ejecutarse de forma eficiente para garantizar una buena experiencia de usuario para satisfacer sus necesidades
Seguridad	El sistema debe mantener segura la información de usuarios, como información crítica	La plataforma maneja datos sensibles principalmente para los residentes, deudas, transacciones. Se implementará un acceso seguro y validación de datos
Mantenibilidad	El sistema debe facilitar futuras mejoras o modificaciones de todo ámbito	La arquitectura modular y el uso de separación por capas permite modificar funciones o módulos sin afectar todo el sistema
Escalabilidad	El sistema debe permitir añadir nuevas funciones o servicios	La arquitectura modular del sistema permitirá incorporar nuevas funcionalidades y servicios externos sin afectar significativamente los módulos existentes.
Disponibilidad	El sistema debe mantenerse operativo y accesible para usuarios	Los usuarios del sistema requieren acceso a la información constantemente
Portabilidad	El sistema debe poder utilizarse desde distintos dispositivos y navegadores (responsive)	Mirador 360° considera el acceso desde distintas plataformas y dispositivos, por lo cual debe ser adaptativo

5.3. Reglas y criterios de evaluación de calidad

Usabilidad

Reglas y criterios

- El sistema debe presentarse con una interfaz de manera ordenada y fácil de usar.
- Los usuarios deben acceder a las funcionalidades en menos de 3 clics
- El sistema debe ser intuitivo y consistente

Métodos de evaluación

- Pruebas heurísticas de usabilidad
- Validación funcional con usuarios simulados
- Evaluación de navegación y experiencia usuario

Seguridad

Reglas y criterios

- El acceso al sistema requerirá autenticación mediante correo y contraseña
- Cada usuario se le asignará permisos asociados a su rol dentro del sistema
- La comunicación entre cliente y servidor será mediante HTTPS
- Los pagos serán procesados mediante servicios externos seguros
- El almacenamiento de información será encriptado

Métodos de evaluación

- Pruebas de autenticación
- Asignación y validación de controles de acceso por rol
- Revisión de conexiones seguras y acceso a información sensible
- Servicios de pago externos confiables

Rendimiento

Reglas y criterios

- El tiempo de respuesta de funciones principales no debe superar los dos segundos
- La carga del panel principal y consulta de deudas deben ejecutarse de manera eficiente
- La conexión con la pasarela de pago no debe superar tiempo de respuestas

- elevados
- El sistema debe soportar múltiples consultas en simultáneo

Métodos de evaluación

- Pruebas de carga
- Simulación de múltiples consultas
- Medición de tiempos de respuesta
- Validación de procesamiento de pagos

Mantenibilidad

Reglas y criterios

- El sistema debe mantener su arquitectura modular basada en capas
- Cada módulo deberá tener responsabilidades específicas y desacopladas
- La estructura del sistema deberá permitir futuras modificaciones

Métodos de evaluación

- Revisión de diagramas UML
- Inspección de estructura de paquetes y componentes
- Validación de separación entre fronten, backend y servicios
- Evaluación de cohesión y acoplamiento entre módulos

Escalabilidad

Reglas y criterios

- La arquitectura modular del sistema permitirá incorporar nuevas funcionalidades e integraciones externas sin afectar significativamente la estructura existente.
- El sistema deberá facilitar futuras ampliaciones de la plataforma mediante módulos desacoplados.

Métodos de evaluación

- Revisión arquitectónica
- Validación de modularidad
- Simulación de incorporación de nuevos módulos o servicios

6. PRINCIPIOS DE DISEÑO APLICADOS

6.1. Propósito

6.2. Principios de diseño (por ejemplo: abstracción, acoplamiento, cohesión, encapsulamiento, modularidad)

PRINCIPIO	DESCRIPCIÓN	APLICACIÓN EN EL SISTEMA
Cohesión	Cada módulo o clase tiene una única responsabilidad bien definida.	Los servicios están diseñados para realizar tareas específicas y no múltiples funciones

7. PROTOTIPO

7.1. Propósito

7.2. Mockups (imágenes con una breve descripción)

7.3. Justificar herramientas de prototipado

8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN

8.1. Propósito

8.2. Lista de verificación

8.3. Análisis y métricas de resultados

9. CONTROL DE VERSIONES

9.1. Propósito

9.2. Control de versión utilizado (justificar el tipo de control de versión utilizado (fecha, semántica o secuencial)

9.3. Justificar herramientas de versionamiento

7. CONCLUSIONES

8. BIBLIOGRAFÍA

9. ANEXOS

9.1. Carta Gantt